



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

INFORME PRÁCTICA 8: ENTRENAMIENTO Y TESTEO DE REDES NEURONALES

RAÚL GONZÁLEZ ACOSTA

Marco Pérez Padilla

C/ Padre Herrera s/n
38207 La Laguna
Santa Cruz de Tenerife. España

T: 900 43 25 26

ull.es



1. Motivación del estudio.....	2
2. Datos y preprocesado.....	2
3. Experimento.....	3
3.1. Modelo y configuración del entrenamiento.....	4
3.2. Diseño experimental.....	4
4. Resultados y análisis.....	5
4.1. Número de sentencias en entrenamiento.....	5
4.2. Tamaño de ventana de contexto.....	6
4.3. Número de épocas.....	7



1. Motivación del estudio

En este informe se pretende estudiar cómo afectan tres factores fundamentales, como son el tamaño del conjunto de entrenamiento, la longitud de la ventana de contexto y el número de épocas, al rendimiento y la generalización de un clasificador tipo Bengio (embedding + suma + MLP) en una tarea binaria sobre letras de canciones.

Por tanto, queremos demostrar si, incrementar la cantidad de datos de entrenamiento mejora de forma consistente la precisión en test, si las ventanas de contexto largas tienden a aumentar el sobreajuste en este modelo de agregación por suma, y aproximadamente cuántas épocas ofrecen un equilibrio razonable entre rendimiento y coste computacional.

Para ello vamos a clasificar canciones por autor/estilo o por categoría A/B según señales léxicas (palabras y tokens frecuentes) y señales secuenciales (orden de versos, progresiones temáticas).

El modelo (tipo Bengio) agrega embeddings por suma, lo que lo hace sensible al contenido léxico pero ciego a la permutación del orden. Por ello resulta imprescindible evaluar cómo varían las métricas al cambiar la cantidad de datos, el tamaño de contexto y el tiempo de entrenamiento.

2. Datos y preprocesado

Para llevar a cabo el estudio disponemos de dos *corpus* distintos: corpusA.txt (clase 0) y corpusB.txt (clase 1), en donde se usan hasta 5000 canciones por clase.



Para la partición train/test, se ha realizado en distintas tandas (50/50, 70/30 y 80/20), hemos usado una semilla fija (42) para la reproducibilidad de estos experimentos.

En cuanto a la tokenización, se conservan los tokens especiales <CS>, </CS>, <EOL>, palabras y signos de puntuación para preservar los límites y la estructura de los versos.

El vocabulario ha sido construido únicamente sobre el conjunto de entrenamiento, haciendo uso de los tokens especiales: <PAD>=0, <UNK>=1; y un umbral con una frecuencia mínima de 2 para filtrar tokens raros.

3. Experimento

En primer lugar, para la extracción de ventanas, en cada canción se toma la primera ventana de longitud “context_size” tokens. Si la canción tiene menos tokens se rellena con <PAD>, en cambio, si tiene más, se trunca al inicio.

A continuación, para la construcción del vocabulario se cuentan frecuencias en el conjunto de entrenamiento; las entradas con frecuencia menor que 2 se etiquetan como <UNK>, y se asigna índice 0 a <PAD> y 1 a <UNK>.

Posteriormente se lleva a cabo el proceso de aleatorización y particionado, donde se barajan las canciones con semilla 42 antes de partir en train/test para asegurar la reproducibilidad.

Finalmente, los resultados se guardaron por combinación de (train_ratio, context_size, epochs) incluyendo métricas train_acc y test_acc para facilitar trazabilidad.



3.1. Modelo y configuración del entrenamiento

Para la arquitectura se usó el siguiente flujo: Embedding (dim=64) → suma de embeddings por ventana de context_size tokens → MLP: Linear(64→128) + ReLU → Linear(128→1).

Para poder calcular la pérdida, se usa BCEWithLogitsLoss. Además se hizo un proceso de optimización usando Adam con tasa de aprendizaje $1e-4$. También se fijaron los siguiente hiperparámetros: batch_size=32, embed_dim=64, hidden=128, seed=42.

Finalmente las métricas reportadas son las siguientes: train_acc (exactitud en entrenamiento), test_acc (exactitud en test) y gap = train_acc - test_acc (medida simple de sobreajuste).

3.2. Diseño experimental

Se realizaron tres barridos independientes con diseño one-factor-at-a-time para analizar el efecto aislado de cada factor:

En el primer barrido, específicamente probando el tamaño de entrenamiento, fuimos variando el porcentaje de entrenamiento/testeo de forma incremental, 50/50, 70/30, y 80/20, siempre manteniendo un context_size de 100, y 10 épocas.

En el segundo barrido, trabajamos el tamaño de contexto, variando el context_size: 30, 100, y 200, siempre manteniendo un porcentaje de entrenamiento/testeo del 80/20, puesto que fue lo que mejores resultados dió en el primer barrido; y epochs=10 épocas.

Finalmente, en el último barrido, probamos el número de épocas, donde variamos con 5, 10, y 20, manteniendo un porcentaje de entrenamiento/testeo del 80/20, y un context_size de 100, ya que es la mejor combinación que probamos.



4. Resultados y análisis

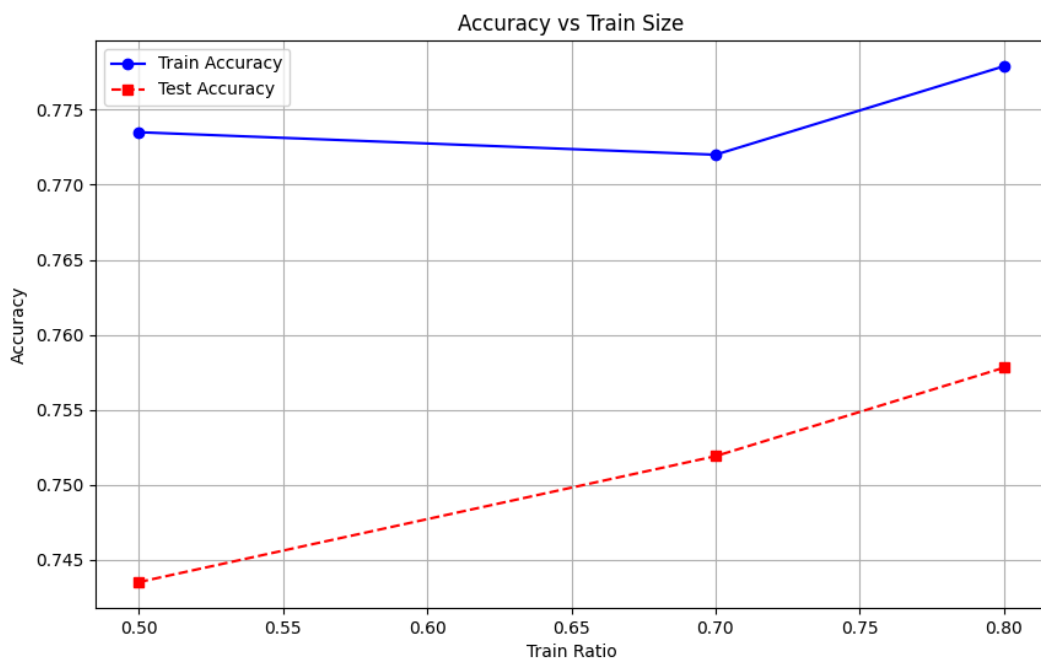
A continuación se muestran los resultados y métricas que obtuvimos después de probar el modelo hasta conseguir la mejor configuración

4.1. Número de sentencias en entrenamiento

Al aumentar la partición de entrenamiento/testeo de 50/50 a 80/20, vimos que el test_acc subía de 0.7435 a 0.7578 y el gap disminuyó de 0.0300 a 0.0201.

La mejora en test indica reducción de varianza y mejor cobertura léxica del dominio cuando hay más ejemplos, y, unido a la estabilidad de train_acc, sugieren que el modelo ya alcanza una capacidad empírica similar en entrenamiento y que la ganancia proviene de generalizar mejor, no de memorizar más.

Por tanto priorizar la adquisición o anotación de más canciones suele ser la vía más segura para mejorar la generalización, sin embargo el coste computacional aumenta con el tamaño del conjunto, por lo que conviene balancear según los recursos disponibles.



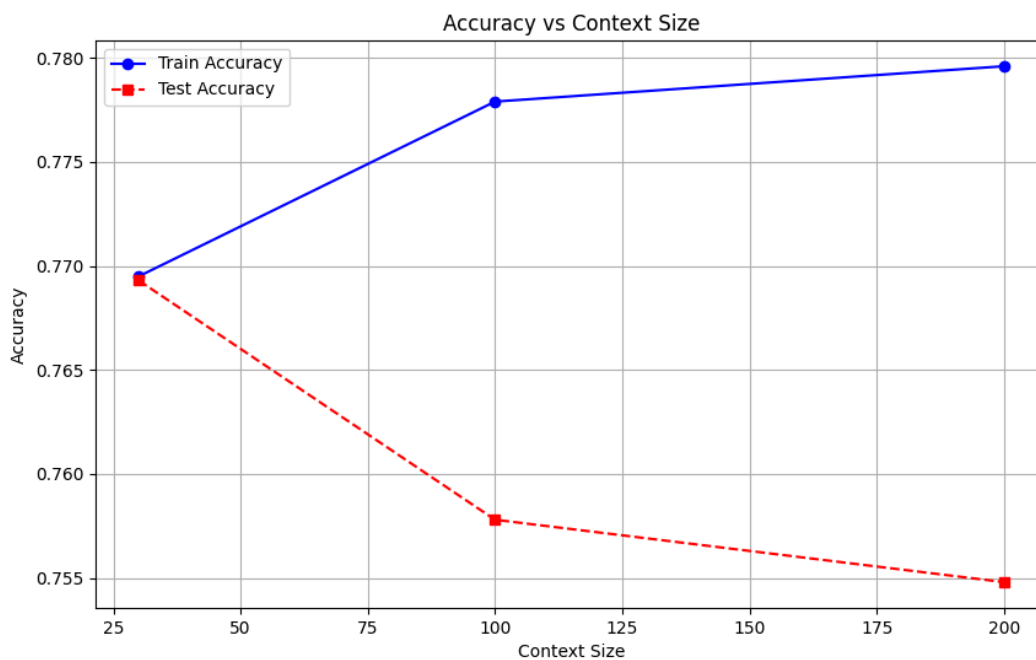


4.2. Tamaño de ventana de contexto

En este caso la variable `test_acc` es máxima para `context_size=30` (0.7693), y decrece para 100 (0.7578) y 200 (0.7548). En cambio, `train_acc` aumenta ligeramente con la ventana, y el gap crece de 0.0002 a 0.0248.

Por tanto, sumar embeddings de ventanas largas introduce más tokens al vector agregado; si muchos de esos tokens son informativos, la performance debería mejorar, pero si son ruidosos la representación agregada empeora la señal discriminativa. Por tanto, el aumento del `train_acc` combinado con la caída en `test_acc` es un indicador clásico de sobreajuste por exceso de información no explotada por la arquitectura.

En modelos de agregación no secuencial, ventanas muy largas pueden ser contraproducentes, y para beneficiarse de contextos largos, conviene usar arquitecturas que modelen orden/atención (RNN, Transformer) o aplicar técnicas de selección/ponderación de tokens.





4.3. Número de épocas

Ahora vemos como `train_acc` crece de forma sostenida entre 5 y 20 épocas (pasando de 0.7621 a 0.7841), pero si entramos más en detalle realmente mejora sustancialmente entre 5 y 10 épocas (pasando de 0.7525 a 0.7578), pero se estanca entre 10 y 20 (pasando de 0.7578 a 0.7584). Por otra parte, el gap aumenta con las épocas.

Por tanto, esto sugiere que en la fase inicial de entrenamiento corrige errores de bias y mejora tanto train como test, pero pasado cierto punto el modelo entra en una fase donde mejora el ajuste a datos de entrenamiento sin transferir esa mejora al test, causando un sobreajuste. El punto de inflexión observado está cerca de 10 épocas para la configuración evaluada.

